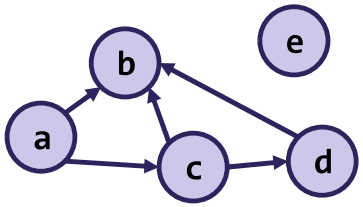
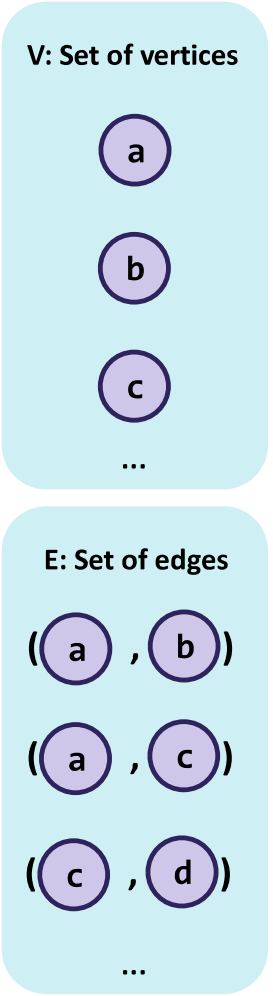
Lecture Outline

- *Review* **Graph Implementations** 
- s-t Connectivity Problem
- BFS and DFS
- Shortest Paths Problem

Review Graph Glossary

- **Graph**: a category of data structures consisting of a set of **vertices** and a set of **edges** (pairs of vertices)
  - **Labels**: additional data on vertices, edges, or both
    - **Weighted**: a graph where edges have numeric labels
  - **Directed**: the order of edge pairs matters (edges are arrows) [otherwise **undirected**]
    - **Origin** is first in pair, **Destination** is second in pair
    - **In-neighbors** of vertex are vertices that point to it, **out-neighbors** are vertices it points to
    - **In-degree**: number of edges pointing to vertex, **out-degree**: number of edges from vertex
  - **Cyclic**: contains at least one cycle [otherwise acyclic]
  - **Simple graph**: No self-loops or parallel edges
- **Path**: sequence of vertices reachable by edges
  - **Simple path**: no repeated vertices
  - **Cycle**: a path that starts and ends at the same vertex
- **Self-loop**: edge from vertex to itself
- **Parallel edges**: two edges between same vertices in directed graph, going opposite directions





Who's that Graph?

AI2's Semantic Scholar is a tool that lets researchers search through a large archive of published papers.

Suppose we want to organize authors and papers in a graph such that it is easy to figure out which authors have worked on papers with other authors (e.g., who are all the authors who co-authored with person X).

Describe the graph structure you would use here.

- What are the vertices? What are the edges?
- Directed or undirected?
- Vertex labels and/or edge labels?
- Will it be a simple graph?

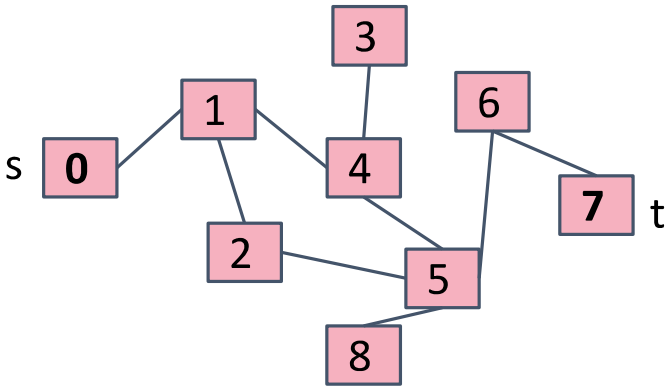
Lecture Outline

- *Review* Graph Implementations
- **s-t Connectivity Problem**
- BFS and DFS
- Shortest Paths Problem

s-t Connectivity Problem

s-t Connectivity Problem

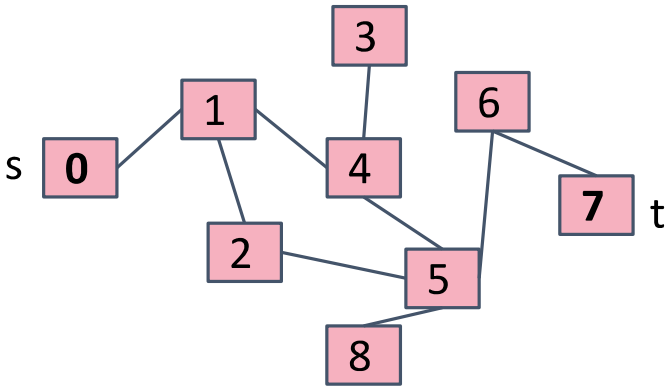
Given source vertex **s** and a target vertex **t**, does there exist a path between **s** and **t**?



- Try to come up with an algorithm for `connected(s, t)`
  - We can use recursion: if a neighbor of **s** is connected to **t**, that means **s** is also connected to **t**!

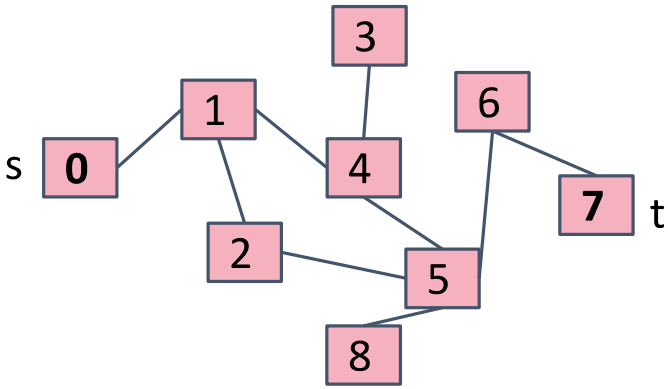
s-t Connectivity Problem: Proposed Solution

```
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    for (Vertex n : s.neighbors) {
      if (connected(n, t)) {
        return true;
      }
    }
    return false;
  }
}
```



What's wrong with this proposal?

```
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    for (Vertex n : s.neighbors) {
      if (connected(n, t)) {
        return true;
      }
    }
    return false;
  }
}
```

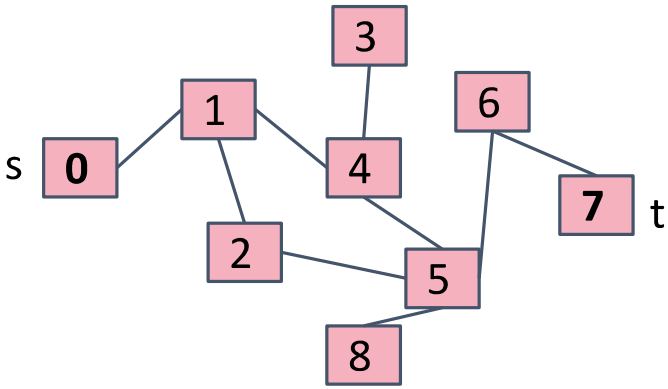


Does 0 == 7? No; if(connected(1, 7) return true;
Does 1 == 7? No; if(connected(0, 7) return true;
Does 0 == 7?

s-t Connectivity Problem: Better Solution

- Solution: Mark each node as visited!

```
Set<Vertex> visited;  // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



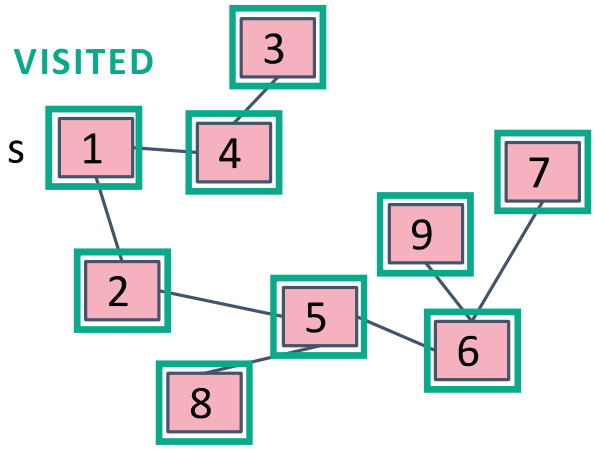
This general approach for crawling through a graph is going to be the basis for a LOT of algorithms!

Recursive Depth-First Search (DFS)

- What order does this algorithm use to visit nodes?
  - Assume order of `s.neighbors` is arbitrary!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```

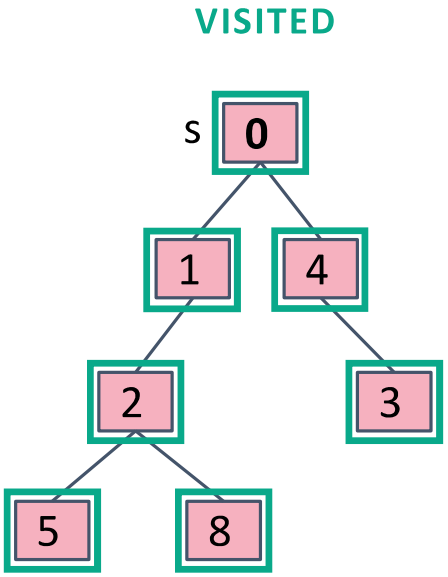
- It will explore one option "all the way down" before coming back to try other options
  - Many possible orderings: {0, 1, 2, 5, 6, 9, 7, 8, 4, 3} or {0, 1, 4, 3, 2, 5, 8, 6, 7, 9} both possible
- We call this approach a **depth-first search (DFS)**



Aside Tree Traversals

- We could also apply this code to a tree (recall: a type of graph) to do a depth-first search on it

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



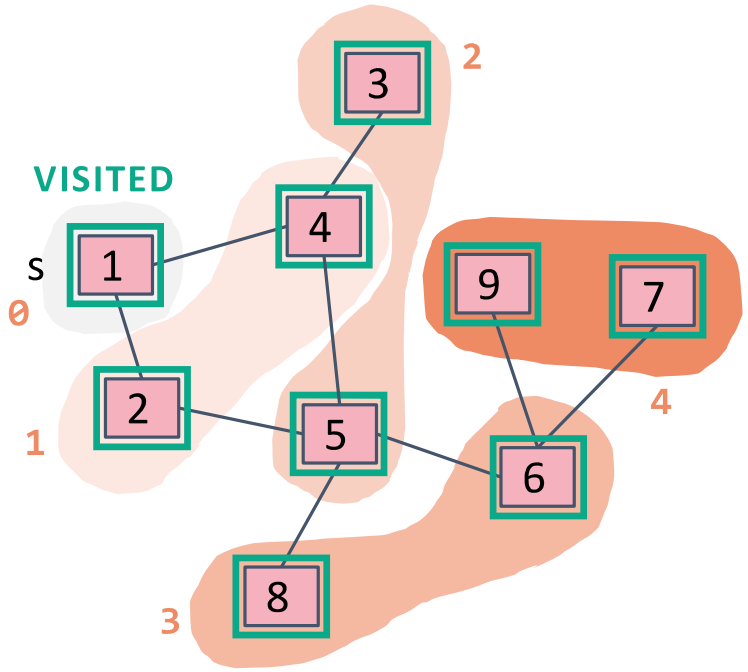
- traversing a binary tree depth-first has 3 options:
  - Pre-order: visit node before its children
  - In-order: visit node between its children
  - Post-order: visit node after its children
- The difference between these orderings is **when we "process" the root** – all are DFS!

Lecture Outline

- *Review* Graph Implementations
- s-t Connectivity Problem
- **BFS and DFS** 
- Shortest Paths Problem

Breadth-First Search (BFS)

- Suppose we want to visit closer nodes first, instead of following one choice all the way to the end
  - Just like level-order traversal of a tree, now generalized to any graph



- We call this approach a **breadth-first search (BFS)**
  - Explore "layer by layer"
- This is our goal, but how do we translate into code?
  - Key observation: recursive calls interrupted s.neighbors loop to immediately process children
  - For BFS, instead we want to *complete* that loop before processing children
  - Recursion isn't the answer! Need a data structure to "queue up" children…

```
for (Vertex n : s.neighbors) {
```

BFS Implementation

Let's make this a bit more realistic and add a Graph

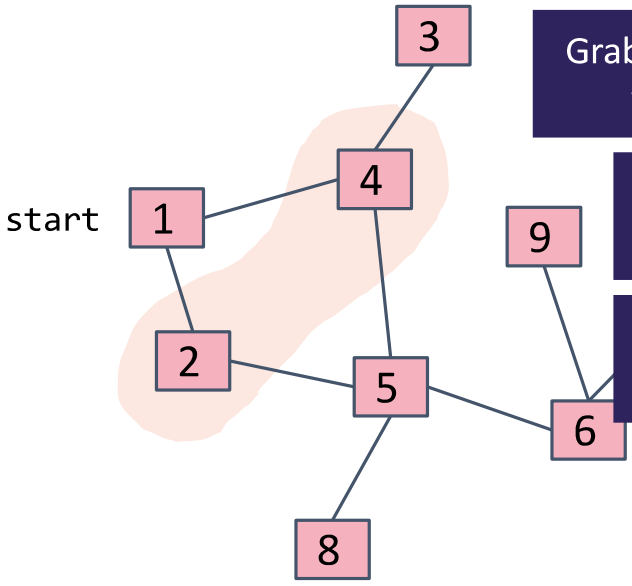
Our extra data structure! Will keep track of "outer edge" of nodes still to explore

Kick off the algorithm by adding start to perimeter

Grab one element at a time from the perimeter

Look at all that element's children

Add new ones to perimeter!

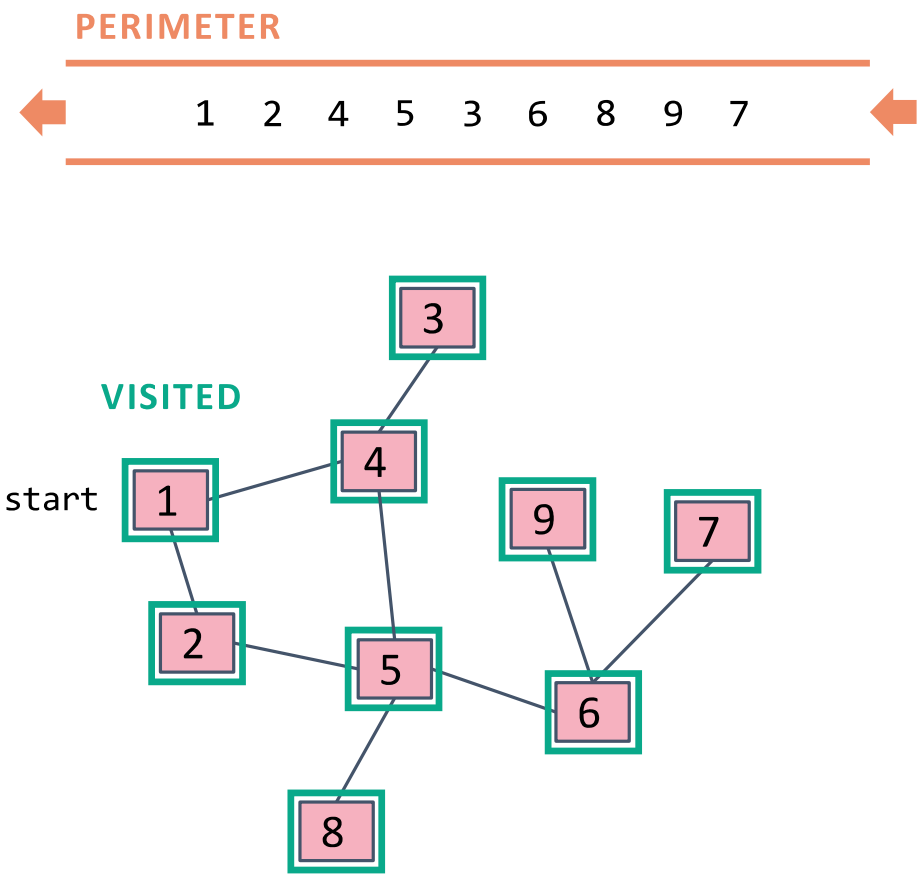



```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

BFS Implementation: In Action

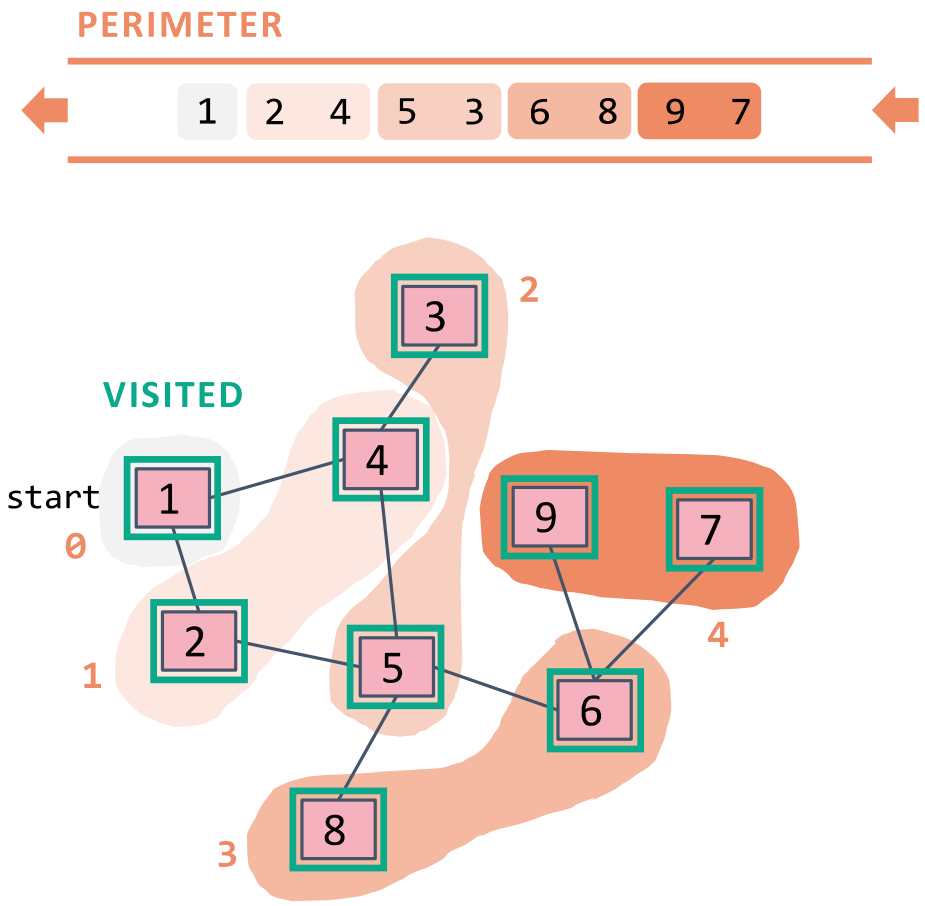



```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

BFS Intuition: Why Does it Work?

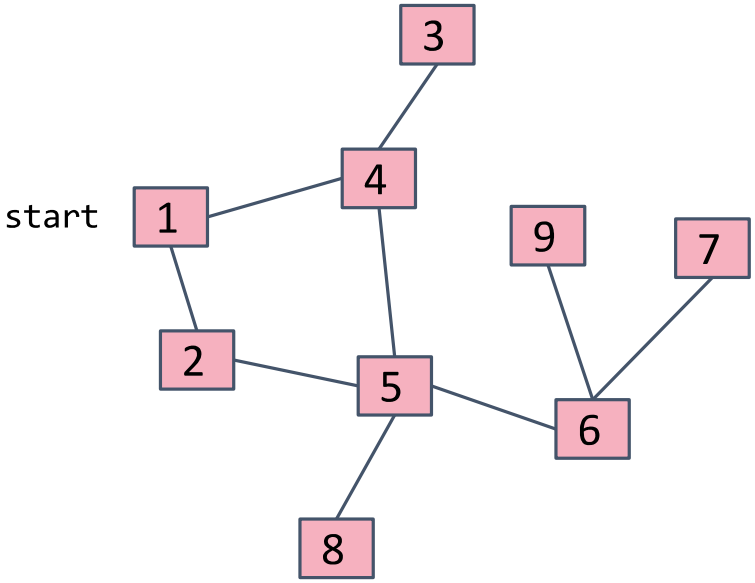


- Properties of a queue exactly what gives us this incredibly cool behavior
- As long as we explore an entire layer before moving on (and we will, with a queue) the next layer will be fully built up and waiting for us by the time we finish!
- Keep going until `perimeter` is empty

```
  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Change Data Structure?

Try to think what would happen if we explored the graph using a Stack for the perimeter instead of a queue.




```
bfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.push(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.pop();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.push(to);
        visited.add(to);
      }
    }
  }
}
```

BFS's Evil Twin: DFS!

Just change the Queue to a Stack and it becomes DFS!
Now we'll immediately explore the most recent child

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Recap: Graph Traversals

- We've seen two approaches for ordering a graph traversal
- BFS and DFS are just techniques for iterating! (think: for loop over an array)
  - Need to add code that actually processes something to solve a problem
  - A *lot* of interview problems on graphs can be solved with **modifications on top of BFS or DFS**! Very worth being comfortable with the pseudocode ☺

Let's Practice Now!

DFS

(Iterative)

- Follow a "choice" all the way to the end, then come back to revisit other choices
- Uses a **stack**!

BFS

(Iterative)

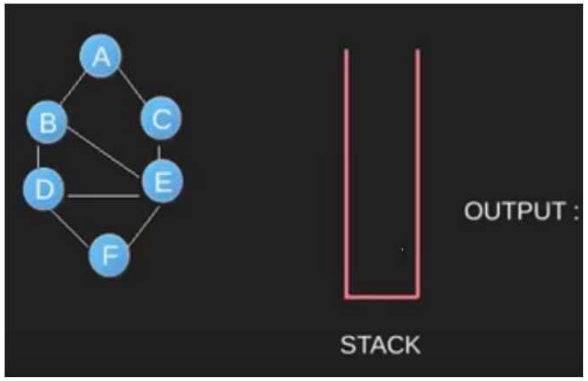
- Explore layer-by-layer: examine every node at a certain distance from start, then examine nodes that are one level farther
- Uses a **queue**!

DFS

(Recursive)

Be careful using this – on huge graphs, might overflow the call stack

DFS Practice

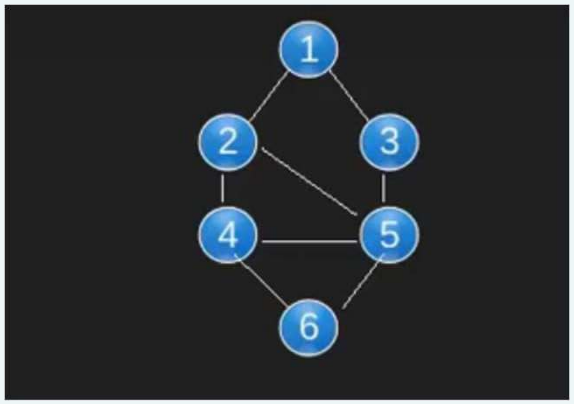



```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

BFS Practice




```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Lecture Outline

- ***Review*** Graph Implementations
- s-t Connectivity Problem
- BFS and DFS
- **Shortest Paths Problem**

The Shortest Path Problem

(Unweighted) Shortest Path Problem

Given source vertex **s** and a target vertex **t**, how long is the shortest path from **s** to **t**? What edges make up that path?

- This is a little harder, but still totally doable! We just need a way to keep track of how far each node is from the start.
  - Sounds like a job for?

Using BFS for the Shortest Path Problem

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Remember how we got to this point, and what layer this vertex is part of

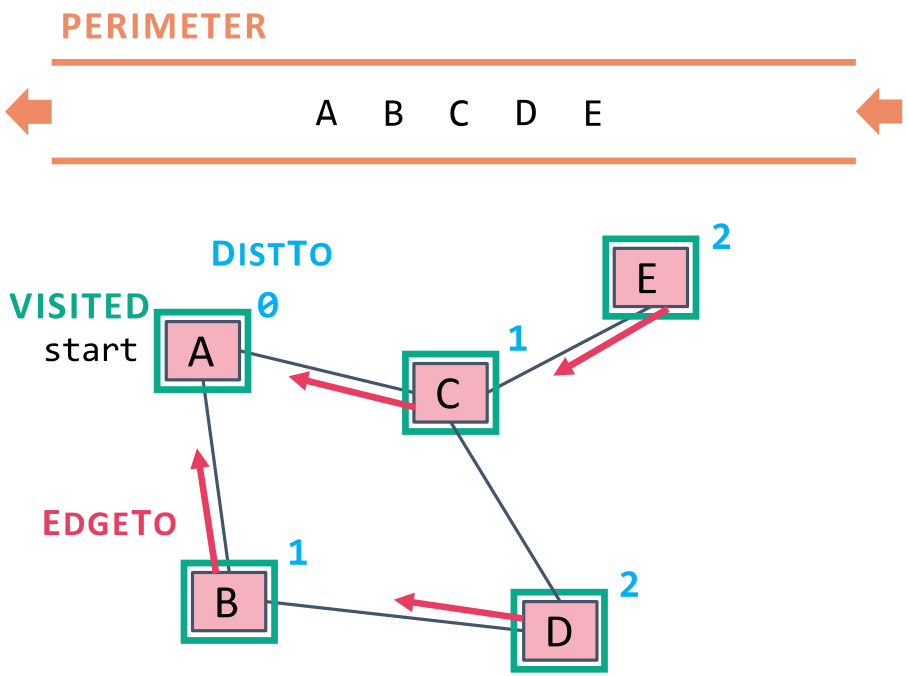
```
  ...
  Map<Vertex, Edge> edgeTo = ...
  Map<Vertex, Double> distTo = ...

  edgeTo.put(start, null);
  distTo.put(start, 0.0);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        edgeTo.put(to, edge);
        distTo.put(to, distTo.get(from) + 1);
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
  return edgeTo;
}
```

The start required no edge to arrive at, and is on level 0

BFS for Shortest Paths: Example



- The edgeTo map stores **backpointers**: each vertex remembers what vertex was used to arrive at it!
- Note: this code stores `visited`, `edgeTo`, and `distTo` as **external maps** (only drawn on graph for convenience). Another implementation option: store them as fields of the nodes themselves

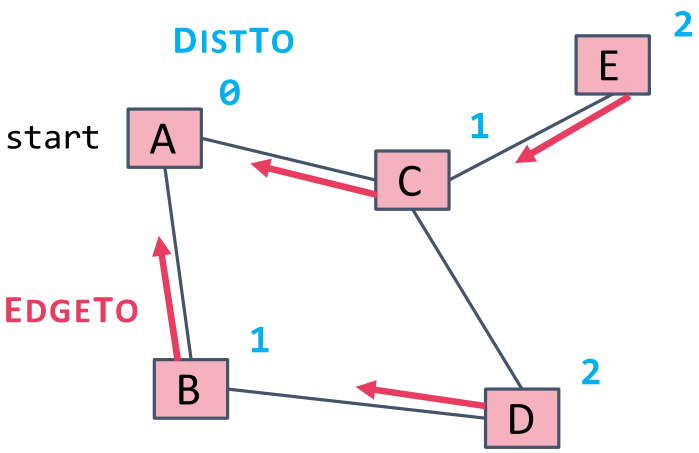
```
  ...
  Map<Vertex, Edge> edgeTo = ...
  Map<Vertex, Double> distTo = ...

  edgeTo.put(start, null);
  distTo.put(start, 0.0);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        edgeTo.put(to, edge);
        distTo.put(to, distTo.get(from) + 1);
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
  return edgeTo;
}
```

What about the Target Vertex?

Shortest Path Tree:



- This modification on BFS didn't mention the target vertex at all!
- Instead, it calculated the shortest path and distance from start to *every other vertex*
  - This is called the **shortest path tree**
    - A general concept: in this implementation, made up of **distances** and **backpointers**
- Shortest path tree has all the answers!
  - **Length of shortest path from A to D?**
    - Lookup in **distTo** map: **2**
  - **What's the shortest path from A to D?**
    - Build up backwards from **edgeTo** map: start at D, follow **backpointer** to B, follow **backpointer** to A – our shortest path is **A → B → D**
- All our shortest path algorithms will have this property
  - If you only care about t, you can sometimes stop early!

Recap: Graph Problems

- Just like everything is Graphs, every problem is a Graph Problem
- BFS and DFS are very useful tools to solve these! We'll see plenty more.

EASY

s-t Connectivity Problem

Given source vertex **s** and a target vertex **t**, does there exist a path between **s** and **t**?

BFS or DFS + check if we've hit t

MEDIUM

(Unweighted) Shortest Path Problem

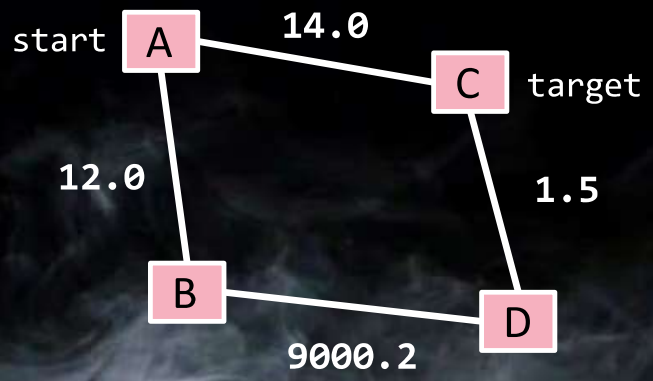
Given source vertex **s** and a target vertex **t**, how long is the shortest path from **s** to **t**? What edges make up that path?

BFS + generate shortest path tree as we go

What about the Shortest Path Problem on a *weighted* graph?

Next Stop Weighted Shortest Paths

HARDER (FOR NOW)



- Suppose we want to find shortest path from A to C, using weight of each edge as "distance"
- Is BFS going to give us the right result here?